PDV WEB · SISTEMA DE PONTO DE VENDA

Tecnologias

Linguagem, frameworks, banco de dados, segurança, testes e infraestrutura do PDV Web — com versões exatas e o motivo de cada escolha.

Documento	DOC-07 · Tecnologias	**Versão**	2.0 — 28/10/2026
Equipe	Hugo de Oliveira Guimarães, Jean Wictor, Cesar Franco		
Situação anterior	O Plano de Projeto citava só Docker e AWS EC2; a Documentação do Projeto trazia o script do banco em **MySQL/MariaDB**. O sistema foi construído em **PostgreSQL** (ADR-001, decisão D-01). Este documento passa a ser a referência de tecnologias.		
Fonte	`pdv-web-api/pom.xml` e a árvore de dependências resolvida pelo Maven		

1. Visão geral da arquitetura

O PDV Web é uma **API REST** em camadas, executada em container Docker numa instância AWS EC2, com banco PostgreSQL. O front-end (telas do Vendedor e do Gerente) consome essa API e é a próxima etapa do projeto.

Camada	Responsabilidade	Tecnologia
Front-end (próxima etapa)	Telas da frente de caixa e da gerência	A definir; consome a API via HTTPS/JSON
Segurança	Autenticação por token e autorização por perfil	Spring Security (OAuth2 Resource Server, JWT)
API (controllers)	Rotas REST, validação de entrada, erros RFC 7807	Spring Web MVC, Bean Validation, Jackson
Serviços	Regras de negócio, transações, auditoria	Spring (transações declarativas)
Persistência	Acesso a dados, consultas e bloqueios	Spring Data JPA, Hibernate, HikariCP
Banco de dados	Armazenamento e restrições de integridade	PostgreSQL 17 + Flyway
Infraestrutura	Empacotamento e execução	Docker, Docker Compose, AWS EC2

2. Backend (`pdv-web-api`)

Tecnologia	Versão	Uso no projeto e motivo da escolha
Java (OpenJDK)	25	Linguagem do projeto (mesma do sistema desktop original). Versão de suporte longo (LTS); usa *records* e *pattern matching*.
Spring Boot	4.1.2	Base da aplicação: configuração automática, servidor embutido e integração entre os módulos abaixo.
Spring Framework	7.0.10	Injeção de dependências, transações declarativas e Web MVC.
Spring Web MVC + Tomcat	Tomcat 11.0.27	Servidor HTTP embutido e controllers REST.
Spring Security	7.1.2	Autenticação stateless por JWT, autorização por perfil em duas camadas e BCrypt para senhas.
Nimbus JOSE + JWT	10.9.1	Assinatura e validação dos tokens JWT (HS256), usada pelo Spring Security.
springdoc-openapi	3.2.0	Documentação interativa da API (Swagger UI em `/swagger-ui.html` e OpenAPI em `/v3/api-docs`), ligada só quando `PDV_DOCUMENTACAO_API=true`.

Tecnologia	Versão	Uso no projeto e motivo da escolha
Spring Data JPA + Hibernate ORM	Hibernate 7.4.13	Mapeamento das tabelas, consultas com filtros e bloqueio de linha (`SELECT … FOR UPDATE`) na venda.
Jakarta Persistence	3.2.0	Padrão de anotações das entidades.
Hibernate Validator (Bean Validation)	9.1.5	Validação declarativa dos dados de entrada, com mensagens em português.
Jackson	3.1.7	JSON em `snake_case` e datas ISO-8601, conforme o contrato da API v2.3.
HikariCP	7.0.2	Pool de conexões com o banco.
Flyway	12.4.0	Criação e versionamento do esquema do banco (migrações V1 e V2) na subida da aplicação.
Driver JDBC PostgreSQL	42.7.14	Conexão com o banco.
Spring Boot Actuator	4.1.2	Endpoint de saúde (`/actuator/health`) para o balanceador e o monitoramento.

3. Banco de dados

SGBD	PostgreSQL 17
Por que PostgreSQL	Já era o banco do sistema desktop; oferece `CHECK` , índice único parcial (um caixa aberto por operador), `TIMESTAMPTZ` , `NUMERIC` exato e bloqueio de linha confiável para vendas concorrentes.
Valores monetários	`NUMERIC(10,2)` no banco e `BigDecimal` no Java — nunca ponto flutuante.
Esquema	Versionado pelo Flyway em `src/main/resources/db/migration` . Ver Diagrama ER (DOC-05) e Dicionário de Dados (DOC-06).

4. Testes e qualidade

Ferramenta	Versão	Uso
JUnit Jupiter	6.0.3	Execução dos testes unitários, de integração e parametrizados
Mockito	5.23.0	Simulação de dependências nos testes unitários de serviço
AssertJ	3.27.7	Asserções legíveis
Spring Boot Test / MockMvc	4.1.2	Testes da camada HTTP e da aplicação completa
Spring Security Test	7.1.2	Simulação de usuários e perfis nos testes de API
Zonky Embedded PostgreSQL	2.2.2 (binários 17.11)	PostgreSQL real nos testes de integração, sem Docker nem instalação
Cucumber	7.34.9	Cenários BDD escritos em Gherkin (português) e executados contra a API
JaCoCo	0.8.15	Cobertura de código; o build falha abaixo de 80% de linhas ou 70% de ramos
PIT (Pitest)	1.30.1	Teste de mutação — mede se os testes detectam erros inseridos no código
Maven Surefire	(gerenciado pelo Spring Boot)	Execução dos testes e relatórios XML/HTML

5. Build, infraestrutura e ferramentas

Tecnologia	Versão	Uso
Apache Maven (via Maven Wrapper)	3.9.16 / 3.3.4	Build e dependências; o wrapper dispensa instalar o Maven
Docker	—	Imagem em duas etapas (build com JDK 25, execução com JRE 25), usuário sem privilégios
Docker Compose	—	Sobe API + PostgreSQL com um comando; configuração por variáveis de ambiente (`.env`)
AWS EC2	—	Hospedagem prevista em nuvem (Plano de Projeto 1.7)
Git	—	Controle de versão; `.gitattributes` garante fim de linha LF nos scripts
GitHub Actions	—	Integração contínua em Linux (Ubuntu + JDK 25): roda toda a bateria e publica os relatórios a cada push (`.github/workflows/ci.yml`)
IntelliJ IDEA	2026.2	IDE de desenvolvimento

> **Situação:** o Dockerfile e o docker-compose foram escritos, mas ainda não foram executados (não havia Docker no ambiente de desenvolvimento). A validação está no roteiro de testes (RT-DEP-01 a 05). A API já foi executada como servidor HTTP real no **modo demonstração** (`demo.sh` / `demo.cmd` , com PostgreSQL embarcado e dados de exemplo).

6. Padrões e convenções adotados

Estilo da API	REST com JSON; contrato definido na Documentação da API v2.3
Formato de erro	RFC 7807 (Problem Details) com `type` , `title` , `status` , `detail` e `invalid_params`
Autenticação	JWT Bearer (RFC 7519), validade de 24 h, revogação no logout
Senhas	BCrypt (fator 10)
Organização do código	Por módulo de negócio (`venda` , `caixa` , `produto` ...) e camadas (controller → serviço → repositório)
Decisões de arquitetura	Registradas em ADR (`docs/arquitetura/ADR-001`)
Documentação de testes	IEEE 829 / ISO/IEC/IEEE 29119-3

7. Sistema desktop legado (`SistemaVendasEmpresa`)

Versão anterior do projeto, mantida como referência e substituída pelo PDV Web.

Tecnologia	Versão	Uso
Java	25	Linguagem
JavaFX (Controls, FXML)	21	Interface gráfica desktop
JDBC + driver PostgreSQL	42.7.2	Acesso direto ao banco
Gson	2.10.1	Leitura do JSON da consulta de CNPJ (BrasilAPI)
Maven + javafx-maven-plugin	0.0.8	Build e execução (`mvnw javafx:run`)